

Introduction to



python

with Application to Bioinformatics

- Day 4

Start by doing today's quiz

Go to Canvas, Modules -> Day 4 -> Review Day 3

~20 minutes

Question 1

```
In [ ]: row = 'sofa|2000|buy|Uppsala'
        fields = row.split('|')
        price = fields[1]
        if price == 2000:
            print('The price is a number!')
        if price == '2000':
            print('The price is a string!')
```

Question 2

```
In [ ]: print(sorted([ 2000, 30, 100 ]))
```

Question 3

```
In [ ]: print(sorted(['2000', '30', '100']))
```

In what ways does the type of an object matter?

- Each type store a specific type of information
 - `int` for integers,
 - `float` for floating point values (decimals),
 - `str` for strings,
 - `list` for lists,
 - `dict` for dictionaries.
- Each type supports different operations, functions and methods.

- Each type supports different **operations**

```
In [ ]: 30 > 2000
```

```
In [ ]: '30' > '2000'
```

- Each type supports different **functions**

```
In [ ]: max('2000')
```

```
In [ ]: max(2000)
```

```
In [ ]: import math  
math.cos(3.14)
```

```
In [ ]: math.cos('3.14')
```

- Each type supports different **methods**

```
In [ ]: 'ACTG'.lower()
```

```
In [ ]: [1, 2, 3].lower()
```

- Each type supports different **methods**

```
In [ ]: 'ACTG'.lower()
```

```
In [ ]: [1, 2, 3].lower()
```

- How to find what methods are available: Python documentation, or `dir()`

```
In [ ]: dir('ACTG') # list all attributes
```

```
In [ ]: dir(str) # list all attributes
```


Questions 4, 5 and 6

```
In [ ]: int('2000') # or float
```

```
In [ ]: float('0.5')
```

```
In [ ]: float('1e9')
```

Question 7

```
In [ ]: values = [1, 0, '', '0', '1', [], [0]]
        for x in values:
            if x:
                print(repr(x), 'is true!')
            else:
                print(repr(x), 'is false!')
```

Question 7

```
In [ ]: values = [1, 0, '', '0', '1', [], [0]]  
for x in values:  
    if x:  
        print(repr(x), 'is true!')  
    else:  
        print(repr(x), 'is false!')
```

- `if x` is equivalent to `if bool(x)`

Question 8: Container types, when should you use which?

- **lists**: when order is important
- **dictionaries**: to keep track of the relation between keys and values
- **sets**: to check for membership. No order, no duplicates.

```
In [ ]: genre_list = ["comedy", "drama", "drama", "sci-fi"]  
genre_list
```

```
In [ ]: genres = set(genre_list)  
genres
```

```
In [ ]: genre_counts = {"comedy": 1, "drama": 2, "sci-fi": 1}  
genre_counts
```

Question 9

- **def**: Starts a new function definition
- **return**: Ends a function
- **print**: Show current state
- **#**: This line is not executed

```
In [ ]: def random_function(message): # starts a new function definition
        if message == "Hi":
            print("Hi")
            #return
        print("This was not a Hi")

random_function("Hi")
```

Converting between strings and lists

- Question 10

```
In [ ]: list("hello")
```

```
In [ ]: '_'.join('hello')
```

TODAY

- Terminal and basic commands
- Basic error handling
- User input in python
- More on functions:
 - return statement
 - positional arguments and keyword arguments
- Comments and documentation
- Pandas - explore your data!

Terminal

- Main commands navigation:
 - cd: **c**hange **d**irectory
 - ls (dir): **l**ist
 - pwd: **p**rint **w**orking **d**irectory
 - tab key
- Other usefull commands: cp, mv, rm, mkdir, cat, echo, clear, touch, grep
- Relative and full paths

More on functions: `return` statement

A function that counts the number of occurrences of 'C' in the argument string.

Functions that `return` are easier to repurpose than those that `print` their result.

```
In [ ]: def cytosine_count(nucleotides):  
        count = 0  
        for x in nucleotides:  
            if x == 'c' or x == 'C':  
                count += 1  
        return count  
  
count1 = cytosine_count('CATATTAC')  
count2 = cytosine_count('tagtag')  
print(count1, "\n", count2)
```

Functions that `return` are easier to repurpose than those that `print` their result

```
In [ ]: cytosine_count('catattac') + cytosine_count('tactactac')
```

```
In [ ]: def print_cytosine_count(nucleotides):  
        count = 0  
        for x in nucleotides:  
            if x == 'c' or x == 'C':  
                count += 1  
        print(count)  
  
        print_cytosine_count('CATATTAC')  
        print_cytosine_count('tagtag')
```

```
In [ ]: print_cytosine_count('catattac') + print_cytosine_count('tactactac')
```

Functions can return multiple values

```
In [ ]: # Modify the function below for returning multiple values  
def cytosine_count(nucleotides):  
    count = 0  
    for x in nucleotides:  
        if x == 'c' or x == 'C':  
            count += 1  
    return count
```

Functions without any return statement returns None

```
In [ ]: def foo():  
        do_nothing = 1  
  
        result = foo()  
        print(f'Return value of foo() = {result}')
```

Functions without any `return` statement returns `None`

```
In [ ]: def foo():  
        do_nothing = 1  
  
        result = foo()  
        print(f'Return value of foo() = {result}')
```

- Use `return` for all values that you might want to use later in your program

Small detour: Python's value for missing values: **None**

- Default value for optional arguments
- Implicit return value of functions without a `return` statement
- `None` is `None` , not anything else (special value).

```
In [ ]: None == 0
```

```
In [ ]: None == False
```

```
In [ ]: None == ''
```

More on functions: Keyword arguments

There are two types of arguments:

- **Positional:** values are passed in the same order as parameters are defined in the function
- **Keyword:** values are passed by parameters names while calling a function

Examples

```
In [ ]: def myFavoriteMovie(pet_number, pet):  
        print(f'I have {pet_number} {pet}')  
        # Positional arguments  
        myFavoriteMovie(2, "dogs")  
  
        # Keyword arguments  
        myFavoriteMovie(pet_number=3, pet="cats")
```

Why do we use keyword arguments?

- It increases the clarity and readability
- Independent on the order of the values
- Very helpfull when functions have many parameters

```
In [ ]: record = 'gene_id INSR "insulin receptor"'
record.split(' ', 2)
```

Why do we use keyword arguments?

- It increases the clarity and readability
- Independent on the order of the values
- Very helpfull when functions have many parameters

```
In [ ]: record = 'gene_id INSR "insulin receptor"'
         record.split(' ', 2)
```

```
In [ ]: record.split(sep=' ', maxsplit=2)
```

The order of keyword arguments does not matter

```
In [ ]: fh = open('../files/fruits.txt', mode='w', encoding='utf-8'); fh.close()
```

```
In [ ]: fh = open('../files/fruits.txt', encoding='utf-8', mode='w'); fh.close()
```

Both positional and keyword arguments can be used (if it is allowed)

```
In [ ]: fh = open('../files/fruits.txt', 'w', encoding='utf-8'); fh.close()
```

But there are some exceptions

```
In [ ]: fh = open('files/recipes.txt', encoding='utf-8', 'w'); fh.close()
```

But there are some exceptions

```
In [ ]: fh = open('files/recipes.txt', encoding='utf-8', 'w'); fh.close()
```

- Positional arguments must be in front of keyword arguments

Restrictions by purpose

Forced to use keyword arguments

```
In [ ]: sorted([1, 4, 100, 5, 6], reverse=True)
```

```
In [ ]: sorted([1, 4, 100, 5, 6], True)
```


Restrictions by purpose

Forced to use keyword arguments

```
In [ ]: sorted([1, 4, 100, 5, 6], reverse=True)
```

```
In [ ]: sorted([1, 4, 100, 5, 6], True)
```

```
In [ ]: sorted(iterable, /, *, key=None, reverse=False)
```

- arguments before `/` must be specified with position
- arguments after `*` must be specified with keyword

How to define functions taking keyword arguments

- Just define them as usual:

```
In [ ]: def format_sentence(subject, value, end):  
        return 'The ' + subject + ' is ' + value + end  
  
        format_sentence('lecture', 'ongoing', '...')  
  
        #format_sentence(subject='lecture', value='ongoing', end='...')
```

How to define functions taking ONLY keyword arguments

```
In [ ]: def format_sentence(*, subject, value, end):  
        return 'The ' + subject + ' is ' + value + end  
  
        format_sentence(subject='lecture', value='ongoing', end='...')  
  
        #format_sentence('lecture', 'ongoing', '...')
```

How to define functions with default arguments

```
In [ ]: def format_sentence(subject, value = "finished", end = "...."):
        return 'The ' + subject + ' is ' + value + end

format_sentence('lecture')
```

How to define functions with optional arguments

- Convention: use the object `None`

```
In [ ]: def format_sentence(subject, value, end='.', second_value=None):  
        if second_value is None:  
            return 'The ' + subject + ' is ' + value + end  
        else:  
            return 'The ' + subject + ' is ' + value + ' and ' + second_value  
  
print(format_sentence('lecture', 'ongoing'))  
  
#print(format_sentence('lecture', 'ongoing', second_value='self-referent'))
```

Exercise 1

- Notebook Day_4_Exercise_1 (~30 minutes)
- Go to Canvas, Modules -> Day 4 -> Exercise 1 - day 4
- Extra reading:
 - <https://realpython.com/python-kwargs-and-args/>
 - <https://able.bio/rhett/python-functions-and-best-practices--78ac1aa>

Documenting and commenting your code

It concerns two group of people:

- users: interested on how it is used
- developers: interested on how it works

Comments:

- mostly for developers. Describes the code
- pound sign (#) is used
- it is short
- tags can be used (BUG, TODO etc)

Documenting:

- mostly for users. Describes the use
- docstrings are used (`""" . . . """`)
- it is quite long

Docstrings

```
In [ ]: def process_file(filename, chrom, pos):  
        """  
        Read a very large vcf file, search for lines matching  
        chromosome chrom and position pos.  
  
        Print the genotypes of the matching lines.  
        """  
        for line in open(filename):  
            if not line.startswith('#'):  
                col = line.split('\t')  
                if col[0] == chrom and int(col[1]) == pos:  
                    print(col[9:])
```

Docstrings

```
In [ ]: def process_file(filename, chrom, pos):  
        """  
        Read a very large vcf file, search for lines matching  
        chromosome chrom and position pos.  
  
        Print the genotypes of the matching lines.  
        """  
        for line in open(filename):  
            if not line.startswith('#'):  
                col = line.split('\t')  
                if col[0] == chrom and int(col[1]) == pos:  
                    print(col[9:])  
  
In [ ]: help(process_file)
```

Docstrings should always be in triple quotes even if they are short and one line

```
In [ ]: import random
def make_list(x):
    # Returns a random list of length x.
    li = list(range(x))
    random.shuffle(li)
    return li
```

```
In [ ]: help(make_list)
```

Comments

Use comments wherever the code is hard to understand

```
In [ ]: my_list[5] += other_list[3]  # explain why you do this!
```

Read more:

<https://realpython.com/documenting-python-code/>

<https://www.python.org/dev/peps/pep-0008/?#comments>

Error handling

Two paths for error handling:

1. Check conditions before performing a task

```
if x_can_be_done:  
    do_this_with_x  
else:  
    error_handling
```

2. Perform the task and deal with errors afterwards

```
try:  
    do_whatever_with_x  
except:  
    error_handling
```

try and except: Check and report errors

Reason for catching errors: Check if I can recover from this error

Try and except basic example

```
In [ ]: try:  
        int("five")  
except ValueError:  
    print("Error: try again")
```

Try and except basic example

```
In [ ]: try:
        int("five")
except ValueError:
    print("Error: try again")
```

Custom and original errors

```
In [ ]: try:
        int("five")
except ValueError as whatever: # Exception is catching all errors
    print("ERROR: There is something wrong")
    print("Original: ", whatever)
```


Catch an error which I can live with (Recover)

Scenario: I am perfect

Input:

- My code is perfect
- The data I use are not perfect, e.g. some entries are wrong

Output:

- I cannot calculate anything because the code will through an error and stop

```
In [ ]: fh = open("../files/bank_statement.txt", "r", encoding = "utf-8")

total = 0

for line in fh:
    expenses = line.strip().split() # split line into list
    store = expenses[0] # save what store
    price = float(expenses[1]) # save the price
    if store == 'ICA': # only count the price if store is
        total = total + price

fh.close()

print('Total amount spent on ICA is: '+str(total))
```

Raise your own error

Scenario: Do not waste my time

- At some point in my code I do not want to continue if the variable is not integer. How can I interrupt the execution?

```
In [ ]: # Accept only integers
my_number=3.2

if type(my_number) is not int:
    print("Oooooohhhh nooooooo")

print("more code follows")
```

User input

The code asks the user for input

- `input()`

```
In [ ]: give_input = input("Give a number: ")  
print(f'You pressed the number {give_input}')
```

Multiple inputs

```
In [ ]: s1=input("s1: ")
        s2=input("s2: ")
        s3=input("s3: ")
        s4=input("s4: ")
        students_list = [s1, s2, s3, s4]
        students_list
```

Type of input

```
In [ ]: number_in=input("Give me a number: ")  
        type(number_in)
```

Type of input

```
In [ ]: number_in=input("Give me a number: ")  
        type(number_in)
```

The type of the input is **ALWAYS** string

Ensure correct type of input (exercise)

We want a script which asks the user to give an integer number. If the input of the user is not integer then inform about that and ask again about an integer number

- Think about it and we will do it together
- Hint: inputs have string type

In []:

Quiz time

Go to Canvas, Modules -> Day 4 -> PyQuiz 4.1

~10 min

Lunch

Pandas!!!

Pandas

- Library for working with tabular data
- Data analysis:
 - filter
 - transform
 - aggregate
 - plot
- Main hero: the `DataFrame` type

Pandas Series

- Pandas Series is a 1D data structure (a column in a table)

```
In [ ]: import pandas as pd
random_names = ["Lena", "Marcus", "Aisha", "Diego", "Sophie", "Jamal"]

list_names = pd.Series(random_names)
list_names
#list_names[2]
```

- Change indexes

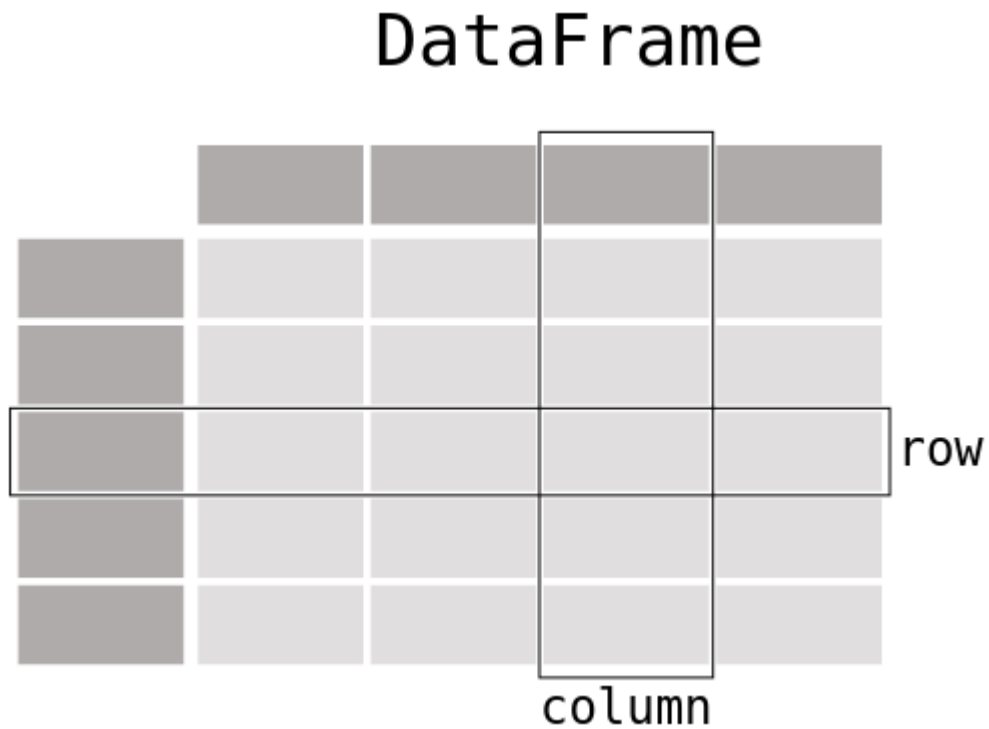
```
In [ ]: new_list_names = pd.Series(random_names, index = range(1, len(random_names)))  
new_list_names
```

- Series from dictionary

```
In [ ]: students = {"student1": "Lena", "student2": "Diego", "student3": "Jamal"}  
students_list = pd.Series(students)  
students_list
```

Pandas DataFrame

- Pandas Dataframe is a 2D data structure. In other words, is a table with rows and columns.



Creating a small DataFrame

```
In [ ]: import pandas as pd
data = {
    'age': [1,2,3,4],
    'circumference': [2,3,5,10],
    'height': [30, 35, 40, 50]
}
df = pd.DataFrame(data)
df
```

```
In [ ]: # add name index  
row_index = ["tree1", "tree2", "tree3", "tree4"]  
df.index = row_index  
df
```

```
In [ ]: # drop index and add a new one  
df = df.reset_index(drop=True)  
df.index = df.index + 1  
df
```

- Locate row

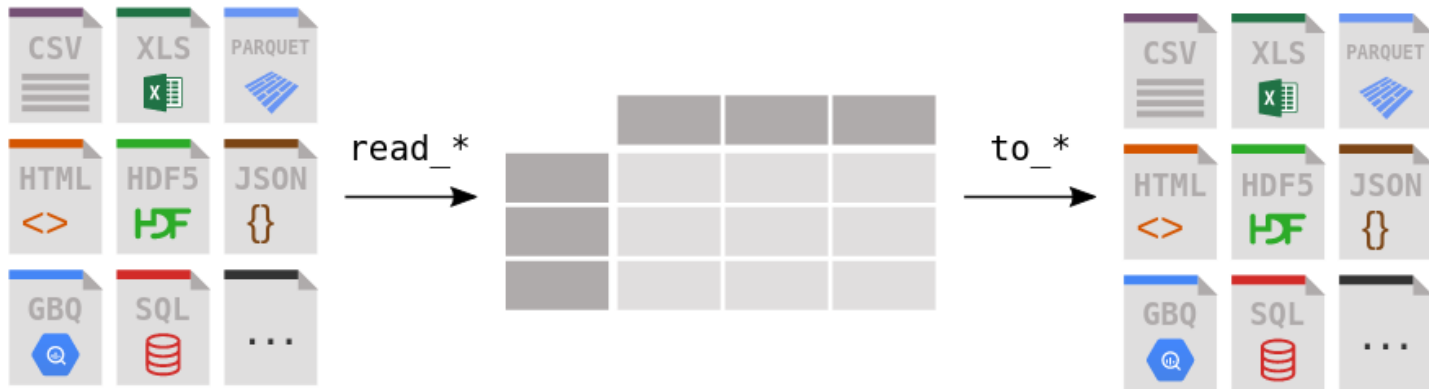
```
In [ ]: df = df.reset_index(drop=True)
df.loc[1]
```

Pandas can import data from many formats

- `pd.read_table` : tab separated values `.tsv`
- `pd.read_csv` : comma separated values `.csv`
- `pd.read_excel` : Excel spreadsheets `.xlsx`

DataFrame can export data to many formats

- `df.to_table()`
- `df.to_csv()`
- `df.to_excel()`



Orange tree data

```
In [ ]: df = pd.read_table('../downloads/Orange_1.tsv')  
df
```

Read larger file

```
In [ ]: df_big = pd.read_csv('../downloads/Orange_1_big.csv')  
df_big
```

Read larger file

```
In [ ]: df_big = pd.read_csv('../downloads/Orange_1_big.csv')  
df_big
```

Do not panic


```
In [ ]: pd.options.display.max_rows
```

Overview of your data, basic statistics

```
In [ ]: df
```

```
In [ ]: df.shape
```

```
In [ ]: df.describe()
```

```
In [ ]: df.max()
```

```
In [ ]: df_big.head(10)
```

```
In [ ]: df_big.tail()
```

```
In [ ]: df_big.info()
```

Selecting columns, rows and cells from a dataframe



Selecting one column

```
dataframe.columnname  
dataframe['columnname']
```

```
In [ ]: df
```

```
In [ ]: df.age
```

```
In [ ]: df['age']
```


Selecting multiple columns

```
In [ ]: df
```

```
In [ ]: df[['age', 'height']]
```

Be careful with the headers of the column

```
In [ ]: import pandas as pd
data = {
    'age ': [1,2,3,4],
    'circumference': [2,3,5,10],
    'height': [30, 35, 40, 50]
}
df_test = pd.DataFrame(data)
df_test
```

```
In [ ]: df_test.age
```

```
In [ ]: df_test['age ']
```

Strip the spaces

```
In [ ]: df_test.columns = df_test.columns.str.strip()  
df_test.age
```

Selecting rows from a dataframe

```
In [ ]: df
```

```
In [ ]: df.loc[0] # select the first row
```

```
In [ ]: df.loc[1:3] # select from row 2 to 4
```

Selecting cells from a dataframe

```
In [ ]: df
```

```
In [ ]: df.loc[[0], ['age']]
```

Run statistics on specific rows, columns, cells

```
In [ ]: df[['age', 'circumference']].describe()
```

```
In [ ]: df['age'].std()
```

Selecting data from a dataframe by index

```
dataframe.iloc[index]  
dataframe.iloc[start:stop]
```

Further reading from pandas documentation: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iloc.html>

```
In [ ]: df  
df.iloc[:,0] # Show the first column  
#df.iloc[1] # Show the second row  
#df.iloc[1,0] # Show the cell of the second row and the first column (yc
```


Creating new column derived from existing column



```
In [ ]: import math
df['radius'] = df['circumference'] / (2.0 * math.pi)
df
```

Pandas concatenate

```
In [ ]: df1 = pd.DataFrame({  
        'age': [1,2,3,4],  
        'circumference': [2,3,5,10],  
        'height': [30, 35, 40, 50]  
    })  
  
df1
```

```
In [ ]: df2 = pd.DataFrame({
    'name': ['palm', 'ada', 'ek', 'olive'],
    'price': [1423, 2000, 102, 30]
})

df2
```

```
In [ ]: df_con = pd.concat([df1,df2])  
df_con
```

```
In [ ]: # index is not reset and it will cause some problems  
df_con.loc[0]
```

Reset index in concatenated dataframe

```
In [ ]: df_con=df_con.reset_index(drop=True) # or pd.concat([df1,df2], ignore_in  
df_con
```

Control the concatenation axis:

- `axis=0` (default): rows concatenation
- `axis=1`: columns concatenation

```
In [ ]: pd.concat([df2, df1], axis=0).reset_index(drop=True)
```

```
In [ ]: pd.concat([df2, df1], axis=1).reset_index(drop=True)
```

Selecting/filtering the dataframe by condition

e.g.

- Only trees with age larger than 100
- Only tree with circumference shorter than 20

Unique values of a column

```
In [ ]: df = pd.read_table('../downloads/Orange.tsv')  
df
```

```
In [ ]: df.Tree.unique()
```

Selecting with condition

```
In [ ]: df[df['Tree'] == 1]
```

```
In [ ]: df[df.age > 500] #reset_index
```

Small exercise 1

- Find the maximal circumference and then filter the data frame by it

```
In [ ]: df
```

Small exercise 2

Here's a dictionary of students and their grades:

```
students = {'student': ['bob', 'sam', 'joe'], 'grade': [1, 3, 4]}
```

Use Pandas to:

- create a dataframe with this information
- get the mean value of the grades

```
In [ ]: students = {'student': ['bob', 'sam', 'joe'], 'grade': [1, 3, 4]}
```

Plotting

- `df.columnname.plot()`
- supports many kinds: line (default), bar, histogram, scatter ...
- more information: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html>

Plotting

- `df.columnname.plot()`
- supports many kinds: line (default), bar, histogram, scatter ...
- more information: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html>

```
In [ ]: small_df = pd.read_table('../downloads/Orange_1.tsv')
        small_df
```

Line plotting

```
In [ ]: import matplotlib.pyplot as plt  
        small_df.plot(x='age', y='circumference', kind='line')
```


Tips: what if no plots shows up?

```
In [ ]: %matplotlib inline
```

Bars Plotting

```
In [ ]: small_df[['age']].plot(kind='bar')
```

Plotting multiple columns

```
In [ ]: small_df[['circumference', 'age']].plot(kind='bar')
```

```
In [ ]: small_df[['circumference', 'age']].plot(kind='bar', figsize=(12, 8), for
```

Plotting histogram

```
In [ ]: small_df.plot(kind='hist', y = 'age', fontsize=18)
```


Plotting box

```
In [ ]: small_df.plot(kind='box', y = 'age')
```

Exercise 2 (~30 minutes)

- Go to Canvas, Modules -> Day 4 -> Exercise 2 – day 4
- **Easy:**
 - Explore the `Orange_1.tsv`
- **Medium/hard:**
 - Use Pandas to read IMDB
 - Explore it by making graphs
- **Extra exercises:**
 - Read the pandas documentation :)
 - Start exploring your own data
- After exercise, do Quiz 4.2 and then take a break
- After break, working on the project